

WOW MY FLIGHT LLC

Custom Travel Sales, Booking & Operations CRM

Complete Product & Development Roadmap

Project: WOW MY FLIGHT — Custom OTA/Travel Agency CRM

Business Model: U.S.-focused online flight sales and travel agency operation

Primary Product: Air-ticket sales, especially competitive/premium/business-class flight sales

CRM Objective: Build a complete internal sales, communication, flight-search, booking, ticketing, finance, QA and management platform.

1. BUSINESS VISION

WOW MY FLIGHT is not looking for a generic CRM such as a normal lead-management system.

We need to build a Travel Sales & Operations Platform specifically for a high-volume flight sales organization.

The system should allow a customer to move through the complete commercial journey:

Lead → Assignment → Contact → Qualification → Flight Search → Quote → Negotiation → Payment → Booking → Ticketing → Travel → After-Sales → Repeat Customer

The CRM must allow management to understand, in real time:

- Where every lead came from
- Who owns the lead
- Who contacted the customer
- What the customer requested
- What flights were offered
- What price was quoted
- What markup/service fee was applied
- Whether the customer paid
- Whether the booking was ticketed
- How much the company made
- Whether follow-up is pending
- Whether an SLA was breached
- Whether QA approved the sale
- What happened after the booking

The CRM therefore becomes the operating system of WOW MY FLIGHT, not simply a database.

2. EXISTING PROTOTYPE / UI REFERENCES

The development team should review the existing prototype routes as the starting point for the user interface and then rebuild/refactor them according to this specification.

Existing prototype references

Super Admin

"Super Admin Dashboard"

(https://flightbookingsystem1.netlify.app/?utm_source=chatgpt.com#/super_admin/dashboard)

Sales Agent

"Agent Dashboard"

(https://flightbookingsystem1.netlify.app/?utm_source=chatgpt.com#/agent/dashboard)

Team Leader

"Team Leader Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/team_leader/dashboard)

Flight Expert

"Flight Expert Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/flight_expert/dashboard)

Integrations

"Integrations Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/integrations)

Expert Team Leader

"Expert Team Leader Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/expert_team_leader/dashboard)

Ticketing Agent

"Ticketing Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/ticketing_agent/dashboard)

Operations

"Operations Dashboard"

(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/operations/dashboard)

Admin

"Admin Dashboard"

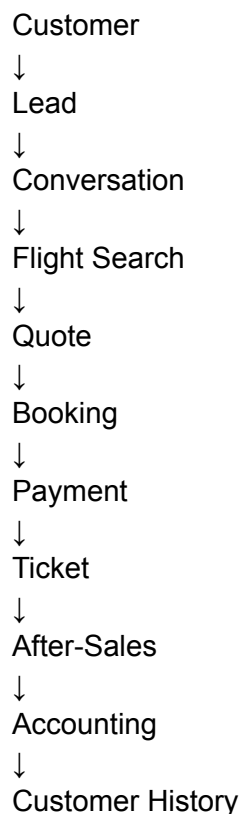
(https://wao-my-flight.netlify.app/?utm_source=chatgpt.com#/admin/dashboard)

These screens should be treated as the current UI concept/reference. The backend, database, permission model, workflows and integrations must be designed properly underneath them.

3. CORE SYSTEM PRINCIPLE

Everything must revolve around a single customer/booking journey.

Master relationship



There must not be separate disconnected records maintained by Sales, Ticketing, Finance and Operations.

There must be one source of truth.

4. SYSTEM MODULES

The CRM should ultimately contain these major modules:

1. Authentication & User Management
2. Role & Permission Management
3. Lead Management

4. Customer 360
5. Sales Desk
6. Communication Center
7. Telnyx Telephony/WebRTC
8. Flight Search
9. Quote Engine
10. Booking Management
11. Ticketing
12. After-Sales
13. Finance & Payments
14. Quality Assurance
15. Team Management
16. Marketing
17. Automation
18. SLA Management
19. Notification Center
20. Supplier Management
21. Integration Management
22. Reporting & Analytics
23. Audit Logs
24. System Administration

5. USER ROLES

Primary roles

Super Admin

Complete system control.

Admin

Operational administration.

Marketing

Campaign and lead-source management.

Finance & Payment

Payments, refunds, reconciliation and financial reporting.

Quality Assurance

Call and booking quality control.

Team Leader

Sales team management.

Expert Team Leader

Complex sales and escalation management.

Ticketing Expert

Ticket issuance, voids, reissues and ticketing queues.

Sales Executive

Lead conversion and customer sales.

Flight Expert

Complex flight research and itinerary support.

Operations Manager

Overall operational control.

Customer Support / After-Sales

Changes, cancellations, refunds, disruptions and customer support.

System/API Manager

External integrations, APIs, webhooks and system monitoring.

Accountant

Accounting and ledger responsibilities.

Read-Only Auditor

Management/audit access without operational modification rights.

6. PERMISSION ARCHITECTURE

This is one of the most important development requirements.

Do NOT build permissions simply as:

Sales = Sales Module

Instead use granular permissions.

Example:

LEAD_VIEW
LEAD_CREATE
LEAD_EDIT
LEAD_ASSIGN
LEAD_REASSIGN
LEAD_DELETE

CUSTOMER_VIEW
CUSTOMER_EDIT

CALL_MAKE
CALL_RECEIVE
CALL_RECORDING_VIEW
CALL_RECORDING_DOWNLOAD

QUOTE_VIEW
QUOTE_CREATE
QUOTE_EDIT
QUOTE_SEND

BOOKING_VIEW
BOOKING_CREATE
BOOKING_MODIFY
BOOKING_CANCEL

TICKET_CREATE
TICKET_VOID
TICKET_REISSUE

PAYMENT_VIEW
PAYMENT_CREATE
PAYMENT_REFUND
PAYMENT_VOID

FINANCE_REPORT_VIEW

USER_CREATE
USER_EDIT
USER_DISABLE

API_CONFIG
SYSTEM_CONFIG

AUDIT_LOG_VIEW

Then apply scope:

- Own Records
- Own Team
- Own Department
- Own Branch
- Own Country
- All Records

For example:

Team Leader

LEAD_VIEW = Own Team

LEAD_EDIT = Own Team

LEAD_ASSIGN = Own Team

BOOKING_VIEW = Own Team

PAYMENT_VIEW = Own Team

PAYMENT_REFUND = NONE

Super Admin can receive unrestricted access.

This architecture follows the security principle of least privilege and deny-by-default, with authorization enforced server-side rather than relying on hidden frontend buttons.

7. SUPER ADMIN CONTROL CENTER

Super Admin should control:

System

- Users
- Roles
- Permissions
- Departments
- Teams
- Branches
- Time Zones
- Currency
- Languages
- Business Settings

- API Credentials
- Integrations
- Notifications
- Workflows

Security

- Login History
- Active Sessions
- Force Logout
- 2FA/MFA
- IP Restrictions
- API Keys
- Audit Logs
- Permission Changes
- Data Access Logs

Financial

- Payment Settings
- Stripe
- Refund Permissions
- Commission Rules
- Markup Rules
- Supplier Settings
- Credit Limits
- Agent Spending Limits

Travel

- GDS
- Consolidators
- NDC Suppliers
- Fare Rules
- Markups
- Service Fees
- Ticketing Rules
- Cancellation Rules
- Queues

Communication

- Telnyx
- Phone Numbers
- Voice
- SMS
- Email
- Templates

- Call Recording

8. SALES EXECUTIVE WORKSPACE

The Sales Executive should have a completely different interface from administration.

The first screen should answer:

«What do I need to sell today?»

My Leads

- New
- Contacted
- Qualified
- Callback
- Quote Sent
- Negotiation
- Payment Pending
- Booked
- Lost

My Tasks

- Call Now
- Callback
- Follow-Up
- Email Follow-Up
- Payment Follow-Up

My KPIs

- Leads Assigned
- Calls Made
- Calls Received
- Talk Time
- Quotes
- Bookings
- Revenue
- Conversion %
- Average Booking Value
- Follow-Up Compliance

9. LEAD MANAGEMENT

Lead sources:

- Website
- Phone
- Email
- Facebook
- Instagram
- Google Ads
- Organic
- WhatsApp
- Referral
- Affiliate
- Partner
- API
- Manual

Customer information

- First Name
- Last Name
- Email
- Mobile
- Country
- Language
- Preferred Contact Method
- Time Zone

Travel requirements

- Origin
- Destination
- Departure
- Return
- One Way
- Round Trip
- Multi-City
- Open Jaw
- Adults
- Children
- Infants
- Cabin
- Flexible Dates
- Airline Preference
- Baggage Requirement

Sales information

- Source
- Campaign
- Assigned Agent
- Team
- Priority
- Lead Score
- Status
- Lost Reason
- Expected Value
- Expected Travel Date

10. LEAD DISTRIBUTION ENGINE

The CRM should support multiple assignment strategies.

Round Robin

Agent A → Agent B → Agent C → Agent D

Skill-Based

Example:

USA → USA Team

Complex Multi-City → Flight Expert

Corporate → Corporate Team

VIP → Senior Agent

After-Hours → Night Team

Capacity-Based

Example:

Agent A = 30 active leads

Agent B = 8

Agent C = 15

New leads should not continue going to Agent A.

Performance-Based

The architecture should allow future routing based on configurable performance criteria.

11. LEAD LIFECYCLE

NEW



ASSIGNED



CONTACTED



QUALIFIED



QUOTE_CREATED



QUOTE_SENT



NEGOTIATION



PAYMENT_PENDING



BOOKED



TICKETED



TRAVEL_COMPLETED

Alternative outcomes:

LOST

SPAM

DUPLICATE

NO_RESPONSE

CANCELLED

Every status change must be audited.

12. DUPLICATE CUSTOMER/LEAD ENGINE

The system should detect duplicates using:

- Email
- Phone
- Name + Phone
- Name + Email
- Travel Requirements

When a potential duplicate is found:

«Potential existing customer/lead found.»

Authorized users can merge records.

The merge operation must preserve the historical activity.

13. CUSTOMER 360

One customer profile should display:

CUSTOMER

- |— Profile
- |— Leads
- |— Calls
- |— Emails
- |— SMS
- |— Quotes
- |— Bookings
- |— Payments
- |— Tickets
- |— Refunds
- |— Documents
- |— Notes
- |— Tasks
- |— Complaints
- |— Audit History

This should become the primary customer-management screen.

14. COMMUNICATION CENTER

All communication should appear in a single timeline.

Example:

10:02 AM — Outbound Call
10:05 AM — Call Ended — 03:12
10:07 AM — Quote Emailed
10:08 AM — SMS Sent
10:15 AM — Customer Replied
10:16 AM — Agent Called

The objective is to eliminate unnecessary switching between systems.

15. TELNYX WEBRTC / SOFTPHONE

The requirement is Telnyx, not Twilio.

Telnyx's WebRTC JavaScript client provides browser-based calling through "TelnyxRTC", including incoming/outgoing calls and browser microphone/device management. Telnyx also states that the WebRTC SDK itself is not sufficient for full call-flow orchestration; backend call-control/webhook architecture is required for advanced flows.

Required functions

- Click-to-Call
- Incoming Calls
- Outbound Calls
- Caller ID
- Call Status
- Hold
- Mute
- Transfer
- Warm Transfer
- Blind Transfer
- Conference
- Call Notes
- Call Disposition
- Recording
- Recording Permissions
- Missed Calls
- Callback
- Call History
- Agent Availability
- Queue
- IVR
- Routing

Incoming call flow

Incoming Number



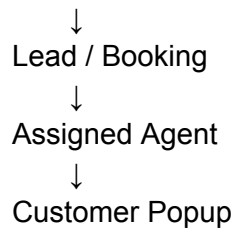
Telnyx



CRM Phone Lookup



Customer



The agent should immediately see customer history.

16. CALL DISPOSITION

Every completed sales call should require a disposition.

Examples:

- Interested
- Not Interested
- Callback
- Quote Sent
- Payment Pending
- Booked
- Wrong Number
- No Answer
- Busy
- Escalated
- Complaint

This data must feed reporting and sales analytics.

17. EMAIL

Do not implement only a "Send Email" button.

Build an actual CRM email system.

Required:

- Inbox
- Sent
- Draft
- Templates
- Scheduled Email
- Attachments
- Threading

- Reply Tracking
- Customer Matching
- Lead Matching
- Booking Matching

A customer email such as:

«"Can you change my return flight?"»

should automatically be associated with the relevant customer, booking, PNR, ticket and team wherever the system can confidently identify the relationship.

18. SMS

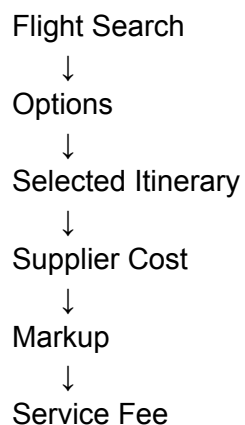
SMS should support:

- Inbox
- Outbound SMS
- Templates
- Automated SMS
- Booking Confirmation
- Payment Reminder
- Callback Reminder
- Flight Change Alert
- Ticketing Notification
- Refund Notification

Every message should be stored against the appropriate customer/lead/booking.

19. QUOTE ENGINE

The quote workflow should be:





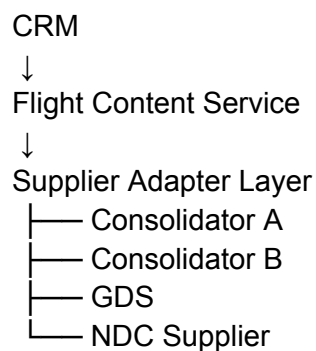
Quote should include:

- Airline
- Flight Number
- Departure
- Arrival
- Duration
- Stops
- Cabin
- Fare Family
- Baggage
- Fare Conditions
- Supplier
- Base Fare
- Taxes
- Supplier Cost
- Markup
- Service Fee
- Discount
- Customer Price
- Expiration Time

20. FLIGHT CONTENT ARCHITECTURE

Do not connect the user interface directly to individual GDS/consolidator APIs.

Create:



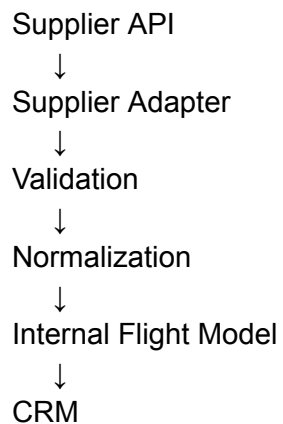
Every supplier should have an adapter.

This allows WOW MY FLIGHT to add or remove suppliers without rebuilding the CRM.

21. SUPPLIER NORMALIZATION

Different suppliers will return different formats.

Therefore:



The CRM should have its own standard flight/booking data model.

Never allow supplier-specific data structures to become the core CRM architecture.

22. FLIGHT EXPERT WORKSPACE

Flight Experts require a more advanced search interface.

Search

- One Way
- Round Trip
- Multi-City
- Open Jaw
- Nearby Airports
- Flexible Dates
- Multiple Passengers

Filters

- Airline

- Alliance
- Stops
- Baggage
- Fare Family
- Departure Time
- Arrival Time
- Connection Duration
- Aircraft
- Cabin

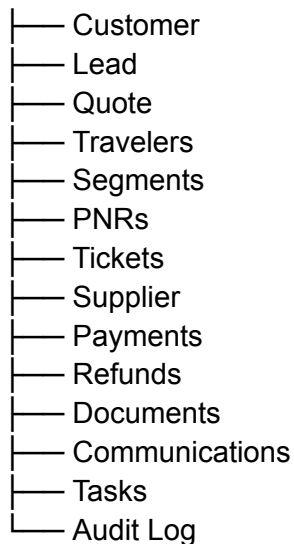
Advanced

- Fare Rules
- Brand Rules
- Baggage Rules
- Seat Availability
- Ticketing Time Limit
- Fare Construction
- Exchange Conditions

23. BOOKING ENGINE

The booking object should be central.

BOOKING



There must be only one authoritative booking record.

24. TICKETING WORKFLOW

Ticketing queue:

PENDING_TICKETING
PENDING_PAYMENT
PAYMENT_CONFIRMED
READY_TO_ISSUE
TICKETING_IN_PROGRESS
TICKETED
FAILED
MANUAL_REVIEW

Ticket record:

- PNR
- Supplier
- Booking ID
- Ticket Number
- Passenger
- Fare
- Taxes
- Commission
- Payment Status
- Ticket Status
- Time Limit
- Issuing Agent
- Date/Time

25. AFTER-SALES

After-sales must be designed from Day 1.

Changes

- Date Change
- Route Change
- Name Correction
- Passenger Correction

Cancellation

- Customer Cancellation
- Airline Cancellation
- Supplier Cancellation

Refund

- Full
- Partial
- Supplier
- Manual

Reissue

- Voluntary
- Involuntary

Disruption

- Schedule Change
- Flight Cancellation
- Misconnection
- Airline Disruption

Every case needs:

- Owner
- Priority
- Status
- SLA
- Notes
- Documents
- Financial impact
- Audit history

26. PAYMENT ARCHITECTURE

Stripe should be treated as an external payment service, not as the CRM's accounting database.

Recommended:

CRM



Payment Service



Stripe



Stripe Webhook



Webhook Validation



Payment Service

↓
Booking
↓
Finance Ledger

Stripe provides webhook endpoints for receiving event notifications, and PaymentIntents are designed around explicit payment confirmation flows.

Never use:

«Browser says payment successful → Booking confirmed.»

Use:

«Stripe event received → signature/event validation → payment state updated → booking workflow continues.»

27. PAYMENT STATUS MODEL

PAYMENT_PENDING
PAYMENT_PROCESSING
PAYMENT_SUCCESS
PAYMENT_FAILED
PAYMENT_CANCELLED
PARTIALLY_PAID
REFUND_PENDING
REFUNDED
PARTIALLY_REFUNDED
CHARGEBACK

Payment status must be controlled by backend business logic.

28. FINANCIAL LEDGER

Every booking should calculate:

Customer Price
- Supplier Cost
- Payment Processing Fee
- Refunds
- Discounts
= Gross Profit

Track:

- Invoice
- Payment
- Refund
- Chargeback
- Supplier Payable
- Commission
- Agent Commission
- Markup
- Service Fee
- Tax
- Ledger Entries

Management must be able to see profitability at:

Booking → Agent → Team → Campaign → Supplier → Overall Company

29. FINANCE PERMISSIONS

Sales

Can:

- Create payment request
- Send payment link
- View payment status

Cannot:

- Refund
- Delete payment
- Modify ledger
- Change accounting records

Finance

Can:

- View payments
- Refund
- Reconcile
- View financial reports

Senior Finance

Can:

- Approve large refunds
- Handle chargebacks
- Make authorized ledger adjustments

Sensitive financial transactions should have server-side authorization and, where appropriate, additional approval controls.

30. QUALITY ASSURANCE

QA should audit both:

Calls + Bookings

Call QA:

Agent

↓

Call

↓

Recording

↓

Transcript

↓

QA Scorecard

Scorecard:

- Greeting — 5
- Customer Verification — 5
- Requirement Understanding — 10
- Product Knowledge — 15
- Fare Explanation — 10
- Accuracy — 20
- Compliance — 15
- Sales Technique — 10
- Closing — 5
- Documentation — 5

Total = 100

31. BOOKING QA

QA should verify:

- Passenger Name
- DOB
- Route
- Date
- Airline
- Fare
- Baggage
- Payment
- Customer Price
- Supplier Price
- Markup
- Ticket
- Confirmation Email
- Agent Notes

This should be capable of identifying costly booking errors before they become customer problems.

32. TEAM LEADER

Team Leader dashboard:

- Team Leads
- Team Bookings
- Revenue
- Conversion
- Calls
- Talk Time
- Follow-Ups
- SLA
- QA
- Revenue/Agent
- Bookings/Agent

Actions:

- Assign Lead
- Reassign Lead
- Monitor Agent
- Coach Agent
- Escalate Lead
- Approve Discount
- Approve Special Pricing
- Monitor Payment Pending

33. EXPERT TEAM LEADER

Expert Team Leader handles:

- Complex Itinerary
- High-Value Booking
- VIP
- Difficult Fare
- Complex Reissue
- Group Travel
- Multi-City
- Corporate
- Escalated Complaint
- Sales Exception

This should be an escalation layer between normal sales and management/operations.

34. MARKETING ATTRIBUTION

Marketing should not measure success only by clicks or leads.

The CRM should track:

Campaign

↓

Lead

↓

Qualified Lead

↓

Quote

↓

Booking

↓

Revenue

↓

Gross Profit

↓

ROI

Store:

- UTM Source
- UTM Medium
- UTM Campaign

- UTM Term
- UTM Content
- Landing Page
- Referral
- Ad ID
- Campaign Cost

35. AUTOMATION ENGINE

Build a reusable workflow engine.

Example:

New Lead

Lead Created



Assign Agent



Notify Agent



Create Task



Start SLA

No Response

No Contact



2 Hours → SMS



24 Hours → Email



48 Hours → Team Leader Alert

Payment Pending

Quote Accepted



Payment Link



30 Min Reminder



2 Hour Reminder



Agent Alert

↓
Team Leader Alert

The exact timing must be configurable by Super Admin.

36. SLA ENGINE

Initial configurable examples:

New Lead → First Contact: 5 minutes

High-Value Lead → First Contact: 2 minutes

Payment Received → Ticketing: 10 minutes

Customer Request → Response: 15 minutes

The system should warn before breach and escalate after breach.

37. NOTIFICATION CENTER

Central notifications:

- New Lead
- Missed Call
- Payment Received
- Payment Failed
- Ticket Time Limit
- Booking Failed
- Customer Reply
- QA Failed
- SLA Breach
- Refund Request
- Flight Schedule Change
- Supplier/API Error

Notifications should be role-aware and permission-aware.

38. OPERATIONS DASHBOARD

Operations should see the health of the entire travel operation.

Live queues

- New Leads
- Pending Quotes
- Payment Pending
- Pending Ticketing
- Failed Ticketing
- Refunds
- Reissues
- Customer Complaints
- Airline Disruptions
- SLA Breaches

Supplier health

- API Status
- Search Errors
- Booking Errors
- Ticketing Errors
- Response Time
- Supplier Availability

39. INTEGRATIONS CENTER

The existing Integrations prototype should become a proper administration console.

Existing Integrations Prototype

Each integration should have:

Provider Name

Environment

API Status

Credentials

Connection Test

Last Successful Request

Last Failed Request

Error Logs

Webhook Status

Rate Limit Status

Enable/Disable

Sandbox/Production

Integrations should include architecture for:

Communication

Telnyx Voice

Telnyx SMS

Email

WhatsApp where applicable

Payment

Stripe

Travel

GDS

Consolidators

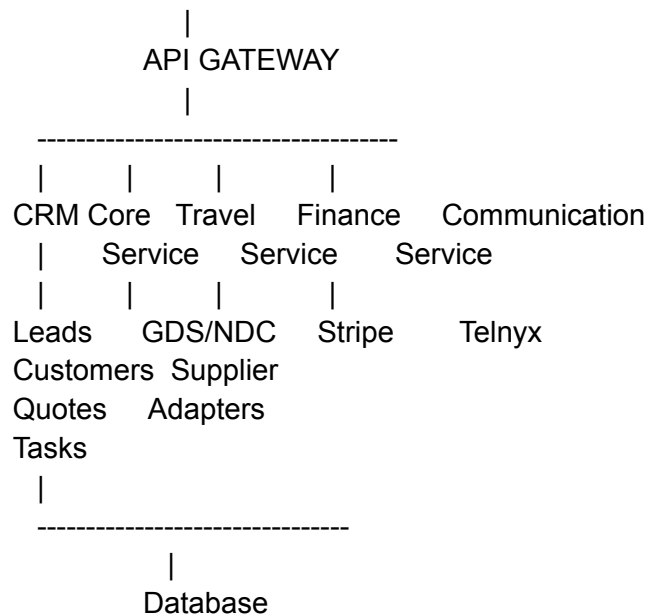
NDC Suppliers

Future suppliers

40. API GATEWAY / SERVICE ARCHITECTURE

Recommended logical architecture:

CRM FRONTEND



The frontend must never communicate directly with sensitive supplier/payment credentials.

41. WEBHOOK ARCHITECTURE

External systems should communicate through controlled webhook services.

Example:

Telnyx

Telnyx

↓

Webhook

↓

Communication Service

↓

Call Event

↓

CRM

Stripe

Stripe

↓

Webhook

↓

Signature/Event Validation

↓

Payment Service

↓

Finance Ledger

↓

Booking

Supplier

Supplier



Webhook/API



Supplier Adapter



Validation



Internal Booking Model



CRM

External providers should never directly modify core CRM database tables.

42. DATABASE MODEL

Initial core entities:

users

roles

permissions

role_permissions

user_roles

departments

teams

branches

customers

customer_contacts

customer_documents

leads

lead_sources

lead_statuses

lead_assignments

lead_events

tasks

notes

calls

call_participants

call_recordings

call_dispositions

emails

email_threads

sms_messages

quotes
quote_items

flight_searches
flight_offers
flight_segments

bookings
booking_passengers
booking_segments
pnrs
tickets

payments
payment_transactions
refunds
chargebacks
invoices
ledger_entries

suppliers
supplier_transactions

after_sales_cases
reissue_cases
refund_cases

qa_reviews
qa_scorecards

campaigns
campaign_leads

notifications

api_requests
webhook_events

audit_logs

43. SECURITY REQUIREMENTS

Mandatory:

MFA/2FA

Fine-Grained Permissions

Server-Side Authorization

Session Management

Password Policies

IP/Device Logging

Audit Logging

Encryption in Transit

Encryption at Rest

Secret Management

API Rate Limiting

Webhook Validation

Database Backups

Disaster Recovery

Data Retention

Secure Error Handling

OWASP specifically recommends authorization checks on every request, deny-by-default, least privilege, centralized authorization, and server-side enforcement.

44. AUDIT LOG

Record:

WHO

WHAT

WHEN

OLD VALUE

NEW VALUE

IP

DEVICE

SESSION

REASON

Examples:

Agent X changed customer price

\$1,820 → \$1,620

Finance User refunded

\$450

Ticketing Agent changed

Pending → Ticketed

Audit logs should be protected from normal modification/deletion.

45. REPORTING

Company

Leads

Contact Rate

Conversion

Revenue

Gross Profit

Average Booking Value

Sales Agent

Leads

Calls

Talk Time

Quotes

Bookings

Conversion

Revenue

Profit

Follow-Up Compliance

Team
Team Leads
Team Conversion
Team Revenue
Team Profit
Agent Performance
Flight
Searches
Search-to-Quote
Quote-to-Booking
Supplier Performance
Booking Failures
Ticketing Failures
Cancellation Rate
Finance
Revenue
Gross Profit
Payments
Refunds
Chargebacks
Supplier Payables
Marketing
Campaign Cost
Leads
Qualified Leads
Quotes
Bookings
Revenue
Gross Profit
ROI